

Workshop description

The workshop exercises cover the design and analysis of parallel algorithms as well as parallel programming on shared and distributed memory architectures. For each problem, you are expected to provide the entire development pipeline for parallel programs. The exercises should be solved in groups.

The workshop session will take place on **May 7, 2026**. Here are the instructions to conduct the workshop exercises.

Instructions

Before the workshop: Solve the problems below, starting with the design of a solution for them. That is, try to specify how you'll solve the problem before you start coding. You'll notice that you have to make some critical choices for their implementation. These choices must be documented, for example, whether you used vectorization, multithreading or multiprocessing, MPI, pipelining or even GPU programming. Make sure that you also implement the code to obtain the average execution time of your algorithms, executing them several times.

Important: You'll need to submit your codes and the necessary files to run them in Moodle before the workshop.

During the workshop: We'll meet in the lecture room and discuss the implementation choices. We'll also try to compare the execution time of the solutions that you implemented.

After the workshop: A single final report per group should be handed in by **May 15, 2026**.

Workshop Exercises

1: Breadth-First Search (BFS) for Large Graphs

Breadth-First Search (BFS) on graphs is a key building block in many real-world applications, including web crawling, social network analysis, recommendation systems, shortest-path algorithms, and more. The focus of this problem is on - designing a sequential BFS algorithm for very large graphs (represented in either adjacency matrix or adjacency list form, depending on memory constraints); analysing the work and depth of this BFS; developing a parallel version for shared memory (e.g., multi-core CPU); and extending the approach for distributed-memory architectures (e.g., MPI or a cluster). You will implement your solutions in code, measure performance, and compare to the theoretical predictions about speedup, efficiency, communication overhead, and so on.

- A. Let G be a graph with V vertices and E edges. You need to explore G starting from a specific source vertex s , visiting all reachable vertices in order of increasing distance from s .
 - I. Explain why the total work done by your BFS procedure is $\Theta(V + E)$. Discuss the “depth” of BFS in a parallel sense, focusing on the number of BFS levels (which often equals the graph’s diameter or a bound on it).
 - II. Implement the sequential queue-based BFS (see Algorithm 1) in a language of your choice. Clearly document how you generate the graph and how you measure the elapsed time.
 - III. Create two graphs, G_1 and G_2 , with G_2 having at least a factor of 10 larger number of nodes than G_1 . You should try to make G_1 as large as possible.
 - IV. Discuss whether BFS can be considered “embarrassingly parallel” or not. Briefly justify your reasoning in terms of data dependencies and communication.
- B. Assume a shared-memory parallel machine (e.g., a multicore CPU or a PRAM-like model with shared global memory).
 - I. Propose a parallel “level-synchronous” BFS algorithm that tracks the current “frontier” (all nodes at the current distance) and explores their neighbours in parallel to form the next frontier. Ensure you indicate how updates to the “visited” or “distance” arrays are handled to prevent data races.
 - II. Create a small illustrative graph (e.g., 6 - 8 nodes) and show a level-by-level BFS DAG. Indicate which tasks run in parallel and how dependencies flow from one BFS level to the next.
 - III. State the total work, parallel depth, expected speedup and efficiency of your BFS in the parallel model
 - IV. Identify key sources of overhead. (e.g., synchronization after each BFS level, load imbalance, memory contention).
 - V. Implement your shared-memory BFS algorithm, using threads, OpenMP, Numba, or similar. Measure its execution time on the same graphs, G_1 and G_2 from 1A. Vary the number of processing elements, record speedup, and compare with the theoretical speedup. Comment on any mismatch.

- C. Now consider that your graph is too large for a single node's memory, or you want to leverage a cluster of interconnected nodes. This section focuses on modifying your BFS to run under a distributed-memory architecture (e.g., MPI).
- I. Describe how you would partition the graph among multiple nodes or processes. For instance, assign disjoint sets of vertices (and their adjacency lists) to each process. Explain how each process will manage its local frontier and how newly discovered vertices are communicated to the processes that own them.
 - II. Provide a brief analysis of the cost of frontier exchange among processes after each BFS level.
 - III. Summarize the overall work in this distributed context. The baseline remains $\theta(V + E)$ for the BFS itself, but it is now subdivided across processes. Discuss how communication or load imbalance can increase the total runtime, leading to sublinear speedup as the number of processes increases.
 - IV. Implement an MPI version of BFS and run it on multiple processes and measure how the performance scales. Compare theoretical predictions with actual timings, focusing on potential bottlenecks such as synchronization at each BFS level or messages exchanged for frontier updates.

```
procedure BFS(G, s):
  Create a queue Q
  for each vertex v in G:
    distance[v] =  $\infty$ 
    visited[v] = false
  distance[s] = 0
  visited[s] = true
  enqueue(s, Q)

  while Q is not empty:
    u = dequeue(Q)
    for each neighbour w of u in G:
      if not visited[w]:
        visited[w] = true
        distance[w] = distance[u] + 1
        enqueue(w, Q)
```

Algorithm 1: Sequential BFS

2: The JPEG image compression algorithm

JPEG is a lossy image compression and encoding algorithm that involves several steps. This problem focuses on the lossy compression part of JPEG. The JPEG algorithm requires transforming a RAW pixel image with values, for example, in the RGB domain to the frequency domain. This is achieved by using some form of the Discrete Cosine Transform (DCT), which is a tool to represent a signal (i.e., the image) as a sum of cosine functions with different frequencies. Let $x = [x_0, x_2, \dots, x_{N-1}]$ be the input vector to the DCT, that is, the signal that we want to transform. Next, let $y = [y_0, y_1, \dots, y_{N-1}]$ be the output vector, which represents

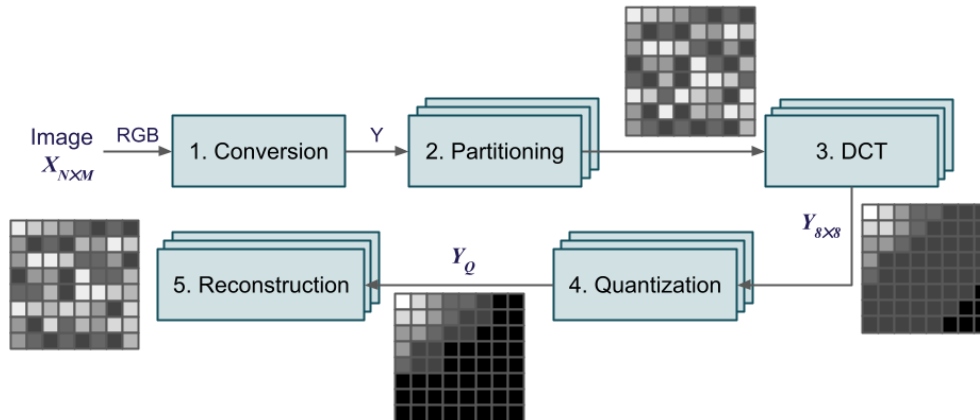
the input x in the frequency domain. The standard DCT, also known as “type-II DCT”, is defined as

$$y_k = 2 \sum_{n=0}^{N-1} x_n \cos \left(\frac{\pi k(2n+1)}{2N} \right)$$

The rationale behind using the DCT for image lossy compression is that it is easier to remove irrelevant data for the human eye from the image in the frequency domain than in the spatial domain. In particular, the high frequency components are not well distinguished by the human eye and, hence, can be removed by simply not adding the cosine functions with those high frequencies. This will result in multiple 0s in the output.

The specific steps that will be considered in this problem are:

1. **Conversion:** Convert the image to grayscale or keep only the luma (i.e., Y) component. The range of values for each pixel is centered to 0.
2. **Block partitioning:** Partition the image into blocks of size $K_h \times K_w$ pixels. The basic JPEG algorithm considers $K_h = K_w = 8$.
3. **Performing the 2D DCT:** The 2D DCT, namely a matrix Y , is obtained by simply performing the one-dimensional DCT operation in both dimensions of the block.
4. **Quantization:** The result is “quantized” by performing the element-wise division of matrix Y by the quantization matrix Q , both of the same **size** $K_h \times K_w$ and rounding the result to the nearest integer. Find the quantization matrices for JPEG online. The final quantized matrix is Y_Q .
5. **Reconstruction:** Perform the inverse DCT, the “type-III DCT,” to recover the image.



You have to investigate the specific implementation details for the 5 steps.

Consider an image of size $N \times M$ pixels and answer/perform the following tasks. **You are not allowed to use functions from the `scipy.fftpack` module.**

- I. What is the work and the complexity of steps 1 to 5?
- II. Create Directed Acyclic Graphs (DAGs) for the parallel implementation of steps 1 to 5.
- III. **Discuss in the group:** Among the methods and packages covered in the course, which one do you expect to have better performance. Rank the methods according to your expected performance for small and large block sizes and for the different steps 1 to 5. Specifically, for $K_h = K_w = 8$ and $K_h = N$ and $K_w = M$. Document the argumentation during the first and second workshop session separately. The methods are listed at the end of the document.

- IV. Implement the DCT algorithm sequentially **and** in parallel using four of the highest ranked sequential and parallel methods for implementation for the two extremes of block sizes. Choose two sequential and two parallel methods. For the large block sizes, you must come up with a method to increase the size of the quantization matrices.
- V. Plot the execution time of the methods for two image sizes. For the parallel implementations in the CPU, find the optimal number of processes for one of your platforms.
- VI. Evaluate the quality of the reconstructed image for different compression factors: just look at the resulting image and assign a score to it: 5 for perfect and 0 for horrible.
- VII. Imagine that your platform is now a distributed cluster and communication occurs through wired links with a data rate of $R = 200$ Mbps. Therefore, transmitting an integer or float of $w \in \{8, 32\}$ bits takes $t_w = w/R$ seconds. Consider this communication cost and analyze the maximum speedup you can get with MPI in a supercomputer.

Methods to reduce execution time covered in the course.

Sequential implementation:

- Explicit vectorization with Numpy
- Just in time compilation (JIT) with Numba
- Numba vectorization

Parallel implementation:

- Numba JIT (parallel=True)
- Multiprocessing
- Threading
- Pool executor: concurrent futures module
- Message passing interface (MPI)
- GPU with cuda or openCL